Lab 08
Object Oriented Programming
BS DS/CY Fall 2025 Morning/Afternoon

Instructions:

- Indent your code properly.
- Use meaningful variable names and follow standard naming conventions.
- Use clear and meaningful prompts/labels for all program input and output.
- Make sure that there are NO dangling pointers or memory leaks in your program.
- Do not discuss your problems with fellow students. If you face any difficulty in understanding, consult your TAs.

Task 1

Create a class called 'ComplexNumber' for performing arithmetic with complex numbers.

Complex numbers have the form

realPart + imaginaryPart * i

where i called iota is $(-1)^{1/2}$ i.e; $i^2 = -1$

For example, 3 + 2i is a complex number, with real part 3 and imaginary part 2.

Use double variables to represent the private data of the class. Provide a constructor function that enables an object of this class to be initialized when it is declared. The constructor should contain default values in case no values are provided.

- Write a member function to overload plus '+' operator to add two complex numbers. Addition of two complex numbers Real part of the first complex number will be added to the real part of the second complex number. And imaginary part of the first complex number will be added to the imaginary part of the second complex number.

  (a + bi) + (c + di) = (a+c) + (b+d)i
- Write a function to overload minus '-' operator to subtract two complex numbers.

  (a + bi) - (c + di) = (a - c) + (b - d)i

- Overload multiplication '*' operator to multiply two complex. Multiplication of two complex numbers Real part of the first complex number will first be multiplied with the second complex number. And the imaginary part of the first complex number will be multiplied with the second complex number.

  (a + bi)(c + di) = ac + bci + adi + bd i 2 = (ac−bd) + (bc+ad)i

- Overload division '/' operator to divide two complex numbers. Division of two complex numbers Express the division as a fraction, then to multiply both numerator and denominator by the complex conjugate of the denominator. This causes the denominator to simplify into a real number:

$$\frac{a+bi}{c+di} = \frac{(a+bi)(c-di)}{(c+di)(c-di)} = \frac{(ac+bd)+(bc-ad)i}{c^2+d^2}$$

$$= \left(\frac{ac+bd}{c^2+d^2}\right) + i\left(\frac{bc-ad}{c^2+d^2}\right).$$

Here is your class declaration.

```
class ComplexNumber
{
private:
        double real;
        double imaginary;
public:
        ComplexNumber(double = 0.0, double = 0.0);
        void display();
        ComplexNumber operator+(const ComplexNumber & ref);
        ComplexNumber operator-(const ComplexNumber & ref);
        ComplexNumber operator*(const ComplexNumber & ref);
        ComplexNumber operator/(const ComplexNumber & ref);
};
```

Task 2

Add the following operators in Matrix class.

Matrix& operator=(const Matrix & ref);
Assignment operator.

Matrix operator+(const Matrix & ref);
Add the calling object and received object and return new Matrix object.

Matrix operator-(const Matrix & ref);
Subtract the calling object and received object and return new Matrix object.

Matrix operator*(const Matrix & ref);
Multiple the calling object and received object and return new Matrix object.

Task 3

Add the following operators in CString class.

CString & operator =(const CString & ref);
Assignment operator.

CString operator+(const CString & s2) const;
Concatenate the calling object and received object and return new CString object.

CString operator+(const char* const s2) const;
Concatenate the calling object and received object and return new CString object.

void operator +=(const CString & s2);
Concatenate the calling object and received object and stores concatenate string in *this.

void operator +=(const char* s2);
Concatenate the calling object and received object and stores concatenate string in *this.

Task 4

Create a class called **Fraction** for performing arithmetic with fractions. Write a driver program to test your class. Use integer variables to represent the private data of the class, the numerator and the denominator. Provide a constructor function that enables an object of this class to be initialized when it is declared.

The constructor should contain default values in case no initializers are provided.

Fraction should be stored in reduced form (use a utility function to reduce) (i.e., the fraction 2/4 would be stored in the object as 1 in the numerator and 2 in the denominator).

You have to implement these functionalities :

1. Add fractions using the operator +
2. Subtract fractions using the operator –
3. Multiply fractions using the operator *
4. Divide fractions using the operator /
5. Compare fractions (six functions) using operators **!=,==, >, <, >=, <=**
6. Increment fractions using post-increment and pre-increment operators ++
7. Decrement fractions using post-decrement and pre-decrement operators - -
8. Take negative of fractions using operator unary –.
9. Compound operators += , -=, *= ,/=

Here is Fraction class declaration.

```
class Fraction
{
private:
        int numerator;
        int denominator;
public:
        Fraction(int = 1, int = 1);
        Fraction operator+(const Fraction &);
        Fraction operator*(const Fraction &);
        Fraction operator/(const Fraction &);
        Fraction operator-(const Fraction &);

        void operator+=(const Fraction &);
        void operator*=(const Fraction &);
        void operator/=(const Fraction &);
        void operator-=(const Fraction &);

        bool operator==(const Fraction &);
        bool operator!=(const Fraction &);
        bool operator<=(const Fraction &);
        bool operator>=(const Fraction &);
        bool operator<(const Fraction &);
        bool operator>(const Fraction &);
        Fraction operator++();
        Fraction operator++(int);
        Fraction operator--();
        Fraction operator--(int);

        Fraction operator-();

        Fraction double();
};
```